

Laboratory Experiment – 5

Aim:

To investigate various instruction addressing modes by analyzing the runtime execution of a C program through the GNU Debugger (GDB), with emphasis on examining CPU register states and disassembled assembly-level instructions.

Theory:

Addressing modes specify the way in which operands are identified and accessed within an instruction. By using GDB commands such as `disassemble` and `info register`, we can inspect the actual assembly instructions generated by the compiler and understand how data is fetched from memory or registers.

1. Immediate Addressing Mode

The operand value is embedded directly inside the instruction. No memory access is needed for the operand.

Assembly examples observed:

- `mov $0xa, %esi` → constant 10 loaded directly into register
- `mov $0x0, %eax` → register initialized to zero
- `movl $0x0, -0x40(%rbp)` → constant 0 written to stack memory
- `cmpl $0x9, -0x40(%rbp)` → immediate 9 compared against stack value

2. Register Addressing Mode

Both source and destination operands reside in CPU registers. This is among the fastest modes since no memory access is required.

Assembly examples observed:

- `mov %rsp, %rbp` → stack pointer copied into base pointer
- `mov %eax, %esi` → value moved between two registers
- `xor %eax, %eax` → register cleared using XOR operation
- `add %rdx, %rax` → register addition performed

3. Direct (Absolute) Addressing Mode

The instruction encodes the precise memory address of the target operand or function.

Assembly examples observed:

- `callq 0x555555550c0 <printf@plt>` → direct call to `printf`
- `callq 0x555555550d0 <__isoc99_scanf@plt>` → direct call to `scanf`
- `callq 0x555555551c9 <displayElements>` → function called by absolute address
- `callq 0x55555555233 <computeTotal>` → direct function invocation

4. Register Indirect Addressing Mode

A register holds the memory address from which data is fetched. The actual operand resides at the address stored in the register.

Assembly examples observed:

- `mov (%rax), %...` → value read from address held in `rax`
- `mov %rax, %rdi` → address passed through register to function
- `mov %rax, %rsi` → pointer value forwarded via register
- `leal -0x30(%rbp), %rax` → effective address loaded into `rax`

5. Indexed / Base Addressing Mode

The effective operand address is derived by combining a base register with a displacement offset. This mode is extensively used in array traversal.

Assembly examples observed:

- `jmp 0x55555555482 <main+99>` → jump to base+offset address
- `jle 0x55555555459 <main+58>` → conditional jump with offset
- `je 0x5555555554b <main+300>` → equality branch to displaced address
- `jmp 0x55555555557 <main+312>` → unconditional indexed jump

6. Relative Addressing Mode

The branch target address is computed relative to the program counter (instruction pointer). This mode appears in conditional and unconditional branch instructions.

Assembly examples observed:

- `jmp 0x55555555482 <main+99>` → forward relative jump
- `jle 0x55555555459 <main+58>` → relative branch on less-or-equal
- `je 0x5555555554b <main+300>` → relative equality jump
- `jmp 0x55555555557 <main+312>` → relative unconditional branch

7. Stack Addressing Mode

Operands are accessed using the stack structure, managed through the stack pointer (`rsp`) and frame pointer (`rbp`) registers.

Assembly examples observed:

- `push %rbp` → current frame pointer saved onto stack
- `mov %rsp, %rbp` → new stack frame established
- `sub $0x50, %rsp` → space allocated on stack for local variables
- `leaveq` → stack frame restored before function return

Source Code:

```
#include <stdio.h>
#define N 10

void displayElements(int data[], int n) {
    printf("\nArray contents:\n");
    for(int i = 0; i < n; i++) {
        printf("%d ", data[i]);
    }
    printf("\n");
}

int computeTotal(int data[], int n) {
    int total = 0;
    for(int i = 0; i < n; i++) {
        total += data[i];
    }
    return total;
}

int locateMax(int data[], int n) {
    int largest = data[0];
    for(int i = 1; i < n; i++) {
        if(data[i] > largest) {
            largest = data[i];
        }
    }
}
```

```

    }
    return largest;
}

int linearSearch(int data[], int n, int target) {
    for(int i = 0; i < n; i++) {
        if(data[i] == target) {
            return i;
        }
    }
    return -1;
}

void arrangeAscending(int data[], int n) {
    for(int i = 0; i < n - 1; i++) {
        for(int j = 0; j < n - i - 1; j++) {
            if(data[j] > data[j + 1]) {
                int hold = data[j];
                data[j] = data[j + 1];
                data[j + 1] = hold;
            }
        }
    }
}

int main() {
    int data[N], target;
    printf("Enter 10 integers:\n");
    for(int i = 0; i < N; i++) {
        scanf("%d", &data[i]);
    }
    displayElements(data, N);
    int total = computeTotal(data, N);
    printf("Total = %d\n", total);
    int largest = locateMax(data, N);
    printf("Largest = %d\n", largest);
    printf("Search value: ");
    scanf("%d", &target);
    int pos = linearSearch(data, N, target);
    if(pos != -1)
        printf("Found at index %d\n", pos);
    else
        printf("Not found\n");
    arrangeAscending(data, N);
    printf("\nSorted:\n");
    displayElements(data, N);
    return 0;
}

```

GDB Session Output:

```

(gdb) info register
rax      0x0                                0
rbx      0x555555555555b0                    93824992236976
rcx      0x0                                0
rdx      0x0                                0
rsi      0x555555555592c0                    93824992252608
rdi      0x7ffff7bf8e10                    140737349099024
rbp      0x0                                0x0
rsp      0x7fffffffdef0                    0x7fffffffdef0

```

```

r8      0xa      10
r9      0x4      4
r10     0x555555556028 93824992239656
r11     0x246    582
r12     0x5555555550f0 93824992235760
r13     0x7fffffffdfb0 140737488347056
r14     0x0      0
r15     0x0      0
rip     0x7ffff7e5a092 0x7ffff7e5a092 <__libc_start_main+243>
eflags  0x246    [ PF ZF IF ]
cs      0x33     51
ss      0x2b     43
ds      0x0      0
es      0x0      0
fs      0x0      0
gs      0x0      0

```

(gdb) disassemble main

Dump of assembler code for function main:

```

0x000055555555541f <+0>: endbr64
0x0000555555555423 <+4>: push    %rbp
0x0000555555555424 <+5>: mov     %rsp,%rbp
0x0000555555555427 <+8>: sub     $0x50,%rsp
0x000055555555542b <+12>: mov     %fs:0x28,%rax
0x0000555555555434 <+21>: mov     %rax,-0x8(%rbp)
0x0000555555555438 <+25>: xor     %eax,%eax
0x000055555555543a <+27>: mov     $0xa,%esi
0x000055555555543f <+32>: lea     0xbd3(%rip),%rdi # 0x555555556019
0x0000555555555446 <+39>: mov     $0x0,%eax
0x000055555555544b <+44>: callq   0x5555555550c0 <printf@plt>
0x0000555555555450 <+49>: movl    $0x0,-0x40(%rbp)
0x0000555555555457 <+56>: jmp     0x555555555482 <main+99>
0x0000555555555459 <+58>: lea     -0x30(%rbp),%rax
0x000055555555545d <+62>: mov     -0x40(%rbp),%edx
0x0000555555555460 <+65>: movslq  %edx,%rdx
0x0000555555555463 <+68>: shl     $0x2,%rdx
0x0000555555555467 <+72>: add     %rdx,%rax
0x000055555555546a <+75>: mov     %rax,%rsi
0x000055555555546d <+78>: lea     0xbb8(%rip),%rdi # 0x55555555602c
0x0000555555555474 <+85>: mov     $0x0,%eax
0x0000555555555479 <+90>: callq   0x5555555550d0 <__isoc99_scanf@plt>
0x000055555555547e <+95>: addl    $0x1,-0x40(%rbp)
0x0000555555555482 <+99>: cmpl    $0x9,-0x40(%rbp)
0x0000555555555486 <+103>: jle     0x555555555459 <main+58>
0x0000555555555488 <+105>: lea     -0x30(%rbp),%rax
0x000055555555548c <+109>: mov     $0xa,%esi
0x0000555555555491 <+114>: mov     %rax,%rdi
0x0000555555555494 <+117>: callq   0x5555555551c9 <displayElements>
....
0x000055555555559f <+384>: retq
End of assembler dump.

```

Conclusion:

This experiment demonstrated how various addressing modes manifest at the assembly level during program execution. Using GDB, we confirmed that the compiler generates instructions utilizing immediate, register, direct, indirect, indexed, relative, and stack addressing modes depending on the nature of each operation. The disassembly and register inspection tools proved essential in identifying these patterns within the compiled C code.